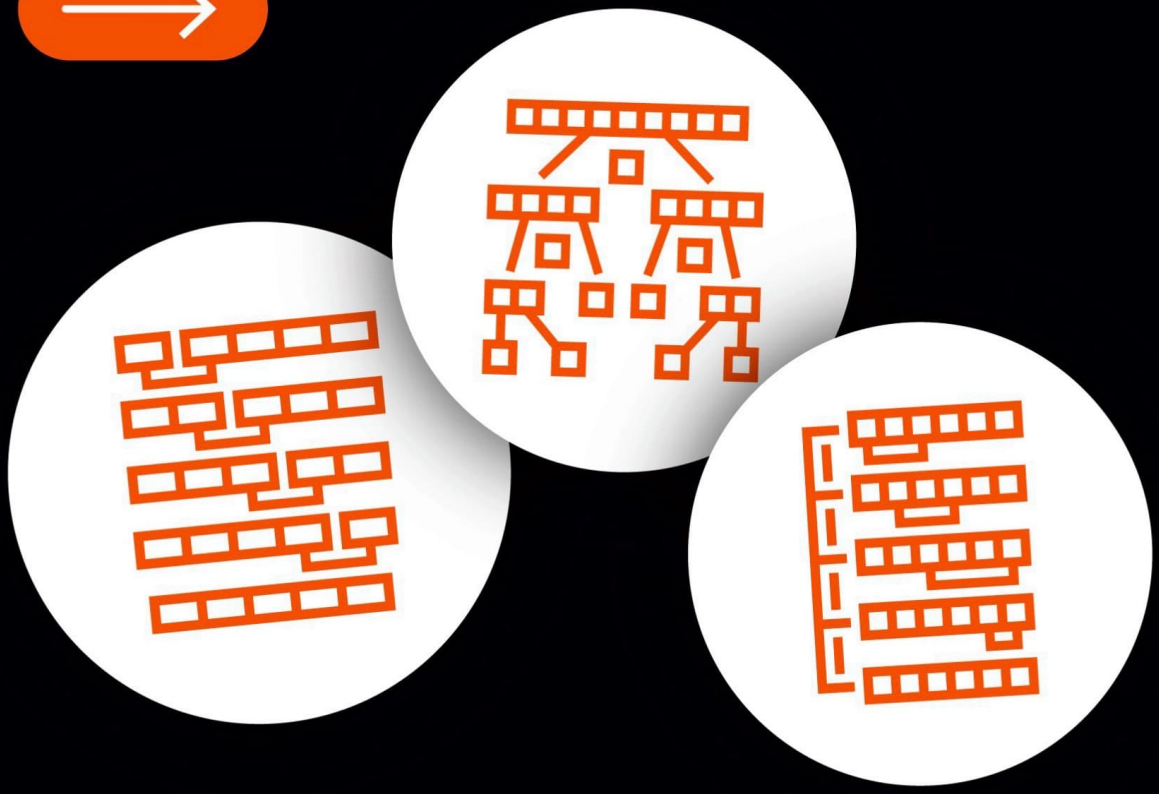
codingninjas

Sorting Algorithm.py

Sorting Algorithm.py

7 Sorting Algorithms

You Can't Skip



Bubble Sort.py

Bubble Sort.py

1. Bubble Sort

Theory:
A simple **comparison-based sorting algorithm** where adjacent elements are compared and swapped if needed. Repeated passes ensure the array becomes sorted. Smaller elements “bubble” up.

Algorithm:

- Compare adjacent elements
- Swap if out of order
- Repeat for all elements
- Stop when no swaps occur

Time Complexity:

- Best: $O(n)$
- Avg/Worst: $O(n^2)$

Merge Sort.py

Merge Sort.py

2. Merge Sort

Theory:

A **Divide and Conquer algorithm** that splits the array, recursively sorts halves, and merges them into a sorted array.

Algorithm:

- Divide array into halves
- Recursively sort both
- Merge sorted halves

Time Complexity:

- Best/Avg/Worst: O(n log n)

Insertion Sort.py

Insertion Sort.py

3. Insertion Sort

Theory:
Builds a sorted array one element at a time by inserting each element into its correct position. Works like sorting cards in hand.

Algorithm:

- Take one element
- Compare with sorted part
- Insert at correct position
- Repeat

Time Complexity:

- Best: $O(n)$
- Avg/Worst: $O(n^2)$

Selection Sort.py

Selection Sort.py

4. Selection Sort

Theory:

A **comparison-based sorting algorithm** that repeatedly selects the **smallest element** from the unsorted portion and places it at the beginning.

Algorithm:

- Find minimum element
- Swap with first unsorted
- Repeat for remaining array

Time Complexity:

- Best/Avg/Worst: $O(n^2)$

Quick Sort.py

Quick Sort.py

5. Quick Sort

Theory:
A Divide and Conquer algorithm that selects a pivot, partitions the array, and recursively sorts subarrays.

Algorithm:

- Choose pivot
- Partition elements
- Recursively sort parts

Time Complexity:

- Best/Avg: $O(n \log n)$
- Worst: $O(n^2)$

Heap Sort.py

Heap Sort.py

6. Heap Sort

Theory:
Uses a **Binary Heap** structure to sort elements. First builds a max heap, then repeatedly extracts the maximum element.

Algorithm:

- Build max heap
- Swap root with last
- Heapify remaining
- Repeat

Time Complexity:

- Best/Avg/Worst: O(n log n)

Counting Sort.py

Counting Sort.py

7. Counting Sort

Theory:
A non-comparison sorting algorithm that counts occurrences of elements and places them in correct positions. Works for limited range integers.

Algorithm:

- Find max value
- Count occurrences
- Compute positions
- Build sorted array

Time Complexity:

- Best/Avg/Worst: O(n + k)